

Evaluating Delivery Strategies: A Simulation Study for We-Doo

Deepak Yadav

School of Computing Information

National College of Ireland

Dublin, Ireland

x21219991@student.ncirl.ie

Abstract—This research presents a simulation study evaluating daily parcel delivery strategies for the 'We-Doo' delivery service company. The customer location data used in this study was generated based on the last four digits of the student identification number, resulting in a map with 100 customer locations. The simulation covers a period of 50 days of delivery to analyze the effectiveness of the proposed strategies. The study employed various algorithms, including the Floyd-Warshall algorithm, to optimize delivery routes and maximize delivery process efficiency. Using a discrete-event simulation approach, the simulation models evening parcel delivery would run in a selected commuter township. Key factors considered in this simulation include parcel distribution, route planning, driving times, customer response times, and the capacity of the electric cargo bike.

Index Terms—Simulation; Parcel Delivery; Optimization; Daily Capacity; Floyd-Warshall Algorithm

I. INTRODUCTION

'We-Doo' emerges as a promising startup aiming to revolutionize delivery services in suburban areas. Their innovative concept involves collaborating with established delivery services, utilizing a central location for package drop-offs. By harnessing local drivers and electric cargo bikes, they ensure efficient parcel distribution directly to customers during evenings.

The core focus of 'We-Doo' is on optimizing logistics and promoting eco-friendly delivery. This distinct approach aims to enhance customer satisfaction while minimizing environmental impact, aligning with sustainable practices.

To validate and refine their unique business model, 'We-Doo' is embarking on a simulation study in a commuter township. This study aims to gauge the potential of planned and sustainable daily deliveries, thereby ensuring the practicality and success of their venture.

The study's primary objective is to assess planned and sustainable daily delivery capacities and suggest which cargo bike range fits better for which pvalues, validating the efficiency of the 'We-Doo' business model. The simulation outcomes will offer vital insights into strategy effectiveness, empowering 'We-Doo' to refine their parcel delivery services with well-informed decisions.

II. LITERATURE REVIEW

The aim is to find the most efficient path to reach customers in a commuter township. This performance features a plethora

of algorithms, each of which resembles a part with its own power, contributing to an epic simulation. The literature review will examine the most popular algorithms in existing research such as Dijkstra's Algorithm, A*, Floyd-Warshall Algorithm and Nearest Neighbor Algorithm.

A. Dijkstra's Algorithm - The Virtuoso of Shortest Paths

Dijkstra's Algorithm is a well-known algorithm for finding the shortest path between nodes in a graph. Edsger Dijkstra, a Dutch computer scientist, invented it in 1956. It explores the nearby nodes of a starting node iteratively, updating the shortest path to each node as it goes. Dijkstra's Algorithm is widely used in a variety of applications, including transportation network route design, network routing protocols, and pathfinding in video games. It was adopted in delivery logistics since it assures finding the shortest path as long as the graph does not contain negative-weight edges [1].

B. A* Algorithm - Guided by Heuristics

The A* algorithm is a pathfinding technique that is used in computer science for finding the shortest route between two points on a graph [2]. It is a heuristic search algorithm that employs a combination of the cost of the trip from the starting node and the expected cost to the goal node to choose the next node to visit [2]. The A* algorithm selects the node with the smallest anticipated total cost [2]. It then produces all of the current node's neighbors and determines their anticipated total cost [2]. The A* method is commonly used in video games, robotics, and other applications that require determining the shortest path between two points. It is regarded as one of the most efficient pathfinding algorithms because it incorporates the benefits of both Dijkstra's algorithm and greedy best-first search [3].

C. Floyd-Warshall Algorithm - Unveiling All-Pairs Shortest Paths

The Floyd-Warshall theory is often used to find the shortest pathways between all pairs in a graph [4]. It is a theory that determines the shortest pathways in a weighted graph [5]. The technique operates by calculating a matrix of shortest-path distances between each pair of network vertices [4]. Regardless of the graph structure, the algorithm has a rigid form, and its complexity is always $O(n^3)$ [4] [6]. As a result,

it is not flexible or adaptive for very big and sparse graphs, and it cannot compete with its alternatives for these types of graphs [7].

Several improvements have been proposed to improve the Floyd-Warshall algorithm's performance. A vectorized version of the technique, for example, has been proposed that makes use of the single instruction multiple data (SIMD) instructions available in Intel architectures [6]. The experimental results reveal that instruction-level parallelism produced by simultaneously processing 8 nodes paired with unrolling gives a speed-up of between 2.3 and 5.2 times over previous blocking methods that just use cache locality [6]. Another study describes two reasonably easy Floyd-Warshall algorithm adaptations for determining all-pairs shortest routes. Both techniques have an expected-case time complexity of $O(n \log n)$ for the class of complete directed graphs on n vertices with arc weights chosen at random from a uniform distribution on [7].

D. Nearest Neighbor Algorithm

[8] The Nearest Neighbor method has been used in a variety of applications, such as logistics management, packet distribution, and delivery route optimization [9]. In fact, the Nearest Neighbor method is widely used to solve the Vehicle Routing Problem (VRP), which entails identifying the ideal routes for vehicles to serve a collection of clients with known orders [9]. According to this study, the VRP algorithm can save 13.14% in distance and 13.17% in cost when compared to other approaches [9]. Generally, this algorithm offers improvements in performance, accuracy, and efficiency in various applications.

III. METHODOLOGY

Following are the steps being followed for the simulation of the delivery strategy. All below steps are in same order as it is used in jupyter file.

A. Parameters

In the given problem, the Parameters for the parcel delivery system are as follows:

- 1) The average number of parcels p received by a customer per day is unknown, and the analysis cover a range of $0.2 \leq p \leq 0.3$, including values outside this range. this research is working with p value as **0.22, 0.265, 0.285, 0.35**
- 2) The cargo bike's performance needs investigation for different ranges r (30km, 35km, 40km, and 45km) corresponding to each chosen p value.
- 3) The time required for driving is estimated based on the path length at an average speed of 15 km/h.
- 4) The handover time to customers includes a call time with an exponential distribution (mean time of 40 seconds) and additional handover time per parcel with an exponential distribution (average of 10 seconds per parcel).
- 5) The cumulative preparation time is 50 seconds per parcel, accounting for route planning, parcel sorting, and cargo-bike packing.

- 6) The day-end procedure for reporting and setting up the cargo bike for charging takes 10 minutes.
- 7) The delivery driver has a planned average working time of 3 hours per working day (18:00-21:00).

B. python library & Utilities

The code employs a range of Python libraries for programming and data analysis, including math functions, random number generation, numerical computing with numpy, data manipulation using pandas, simulating discrete event systems via simpy, and performing time-related tasks. Interactive data visualization is facilitated by plotly, while scipy supports scientific and statistical analysis. For static data visualization, matplotlib's pyplot is utilized. These libraries collectively enable various essential tasks such as mathematical operations, simulation, data manipulation, and visualization.

Additionally, the research code employs custom functions for data visualization and analysis. Notably, the `plotMap` function allows plotting geographical data, representing nodes and connections with various styles and annotations. The `histplot` function facilitates analyzing data distributions through histograms, PDF curves, and mean/standard deviation indicators. The `dailyPlot` function aids in displaying trends over time with linear fits. The `barPlot` function generates bar charts for counting data. Moreover, helper functions `dist` and `pathLength` assist in geometric calculations, while `day` and `timestamp` handle time-related data conversions and formatting.

These Python libraries and custom functions have been crucial in performing complex calculations, visualizing data, and conducting statistical analysis, enhancing the overall effectiveness of the research.

C. Data Generation

This research employed the `generateData(seed=9991)` function to execute the simulation delivery system. This function was used to create two pickle files containing customer nodes represented in a 2D diagram for two scenarios: one with 5 customers and another with 100 customers. Subsequently, these pickled files were utilized to facilitate the simulation process.

Moreover, the `generateParcels()` function was utilized to generate the daily delivery demand for each customer independently. This function employs an exponential distribution with an arrival rate determined by the probability denoted as p . Along with the probability parameter, the function accepts additional input parameters such as the number of days for which delivery demands should be generated (`days`), the total number of customers (`customers`), and optional flags (`plot` and `log`) to enable histogram plotting and delivery details printing. The output of this function is a list of lists, with each inner list representing the customers who anticipate deliveries on each respective day.

By employing the `generateParcels()` function, the research enables analysis and simulation of parcel delivery patterns for a specified period and a given customer base.

This approach provides valuable insights into the dynamics of parcel delivery within the simulated system.

D. Shortest Path Finding

After generating customer location and parcel data, the researcher aims to find the shortest path from the delivery center to the customers. The author utilized the function $FW(M)$ to implement the Floyd-Warshall algorithm for all-pairs shortest path calculations in a given graph, represented by vertices V and edges E . The resulting matrices d and p store the shortest distances and paths between each pair of vertices, respectively.

In addition to this approach, the researcher uses two other methods `createLoop()` and `createLoopH()`. The first method, `createLoop()`, employs Integer Linear Programming (ILP) with the PuLP library to optimize a loop path for visiting all customers in the minimum distance. On the other hand, the second method, `createLoopH()`, utilizes a heuristic approach to iteratively construct a loop path by strategically inserting customers to improve overall distance.

E. Recorder Class - Python Data Recording and Visualization Utility

The Recorder class in Python serves as a versatile data recording and visualization utility for simulations. It efficiently records and tracks various simulation events, including driver work schedules, tour lengths, parcel deliveries, and delays. It also generates insightful visualizations, such as histograms and line plots, to present simulation outcomes effectively.

1) *Initialization and Data Frames:* Upon initialization, the class receives essential parameters like environment, driver and bike counts, days of simulation, and parcel numbers. The class then creates data frames to store daily records and parcel-specific information.

2) *Event Recording:* Throughout the simulation, methods within the Recorder class log and record relevant events, such as driver work timings and parcel deliveries. It provides methods like `recordDriverBeginsWork`, `recordDriverEndsWork`, `recordTourLength`, `recordParcelsLeftOver`, `recordParcelArrival`, `recordParcelDelivery`, and `recordParcelCancelled` for this purpose.

3) *Statistics and Finalization:* Upon simulation completion, the class aggregates and calculates statistics, filling in any missing delivery data. The method `finish` finalizes the recorded data and prepares it for analysis.

4) *Visualization:* The Recorder class leverages the `histplot` and `dailyPlot` functions (previously defined) to plot and visualize key metrics, including working time, tour length, left-over parcels, and delivery delays. It also generates a table to display the distribution of parcel delivery delays using the Plotly library.

By offering an intuitive and powerful toolkit for data recording and visualization, the Recorder class significantly enhances the analysis and comprehension of simulation results.

F. Parcel Class

The Parcel class represents a parcel in the delivery system. It tracks the status and timing of various events related to parcel handling. The class contains functions that update the parcel's status during different stages of the delivery process, such as being in transit to the distribution center, being out for delivery, being returned from delivery, etc. The Parcel class also maintains a REGISTER list to keep track of all created parcel instances.

The parcel undergoes a series of stages in the delivery process. Initially, it is packed and sent to the distribution center. At the distribution center, it is sorted and prepared for transfer to different delivery centers. From the delivery center, a driver takes charge of the parcel, arranging it for daily delivery. Once the driver loads the parcel onto his bike, it is marked as "out for delivery." The driver then heads to the customer's location, and upon reaching there, if the customer is available to accept the parcel, it is handed over to the customer.

However, if the customer cancels the order, the parcel is returned to the distribution center. Similarly, parcels that remain undelivered due to the customer's absence are also sent back to the delivery center. The entire flow of these stages is visually depicted in the event graph as shown in Figure 1.

The event graph provides an overview of how parcels progress through various states during the delivery process, reflecting the dynamic nature of the system and facilitating an understanding of the parcel's journey from packing to final delivery or return.

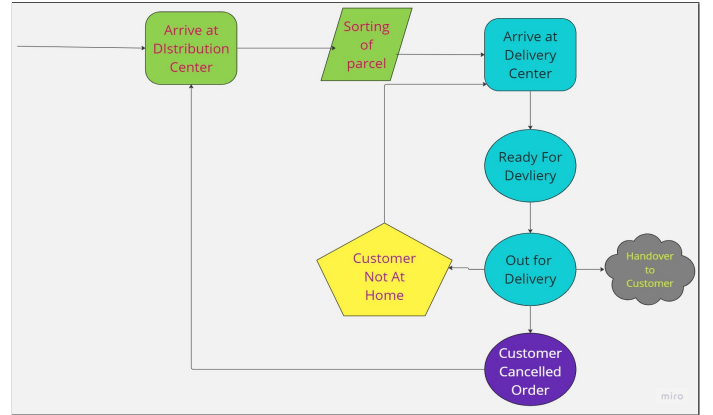


Fig. 1. Event Graph for Parcel

G. Customer Class

The Customer class represents a passive customer in the delivery system. It is associated with a specific location and has a given probability q of being at home. The class includes functions to handle different stages of customer interaction, such as when the customer is not at home, when the customer responds to the delivery attempt, and when the customer's parcel is returned to the delivery center due to non-response. The Customer class ensures that there is only one customer instance per location using a factory method. It also maintains

a REGISTER list to keep track of all created customer instances.

H. Driver Class

The `Driver` class simulates the behavior of delivery drivers. It handles the driver's work schedule, departure for delivery, driving to customers, parcel deliveries, and returning to the delivery center. The class maintains different stages during the delivery process, including when the driver arrives for work, leaves for delivery, drives to the customer's location, arrives at the customer's location, etc. It also handles scenarios where customers respond or do not respond to delivery attempts within the expected response time. The `Driver` class records relevant events and interacts with the `Customer` class to complete the delivery process. Figure 2 displays the whole event process diagram for the driver.

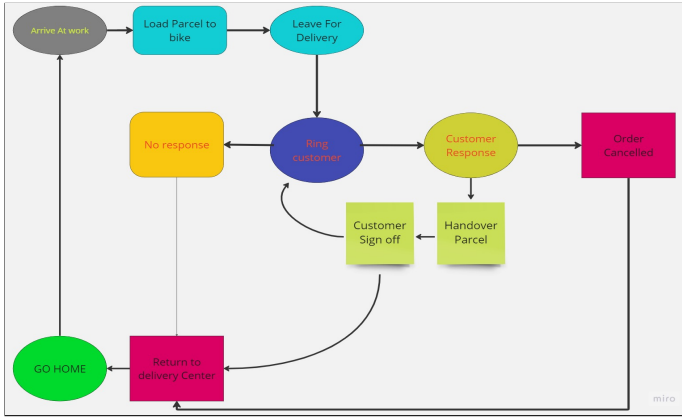


Fig. 2. Event Graph for Driver

I. DeliveryCenter Class

The `DeliveryCenter` class represents the central distribution center in the delivery system. It manages the acceptance of parcels, schedules parcels for delivery using the `createLoop` or `createLoopH` methods, and rearranges left-over parcels. The class maintains lists for parcels that are ready for delivery, parcels out for delivery, and parcels returned to the delivery center. The `DeliveryCenter` class also tracks customer destinations and plans delivery tours to optimize delivery routes using the specified heuristic. Additionally, it includes methods to accept parcels, send parcels for delivery, and return parcels at the end of the day. Figure 3 shows the event graph for the delivery center.

J. Run simulation

The Jupyter file is organized into four distinct sections, each corresponding to different values of the parameter "pValue." These sections are labeled as "pValue = 0.22," "pValue = 0.265," "pValue = 0.285," and "pValue = 0.35." Within each section, the simulation is executed multiple times, varying the value of another parameter "r" (radius) at different levels: 30 km, 35 km, 40 km, and 45 km for each specific pValue. **Total 16 simulation is performed.**

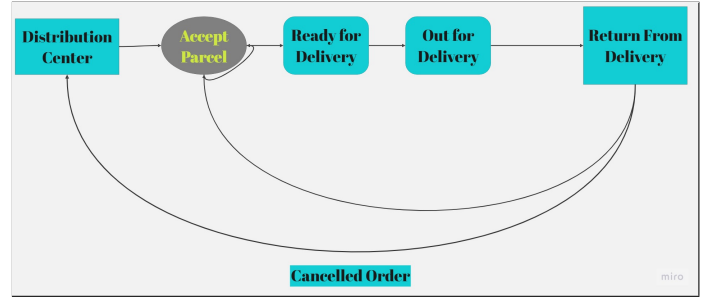


Fig. 3. Event Graph for Delivery center

Once the simulations are performed, the outputs are then analyzed in separate subsections using the visualization methods discussed earlier. These visualization techniques include histograms and line plots, among others. By utilizing these visualization methods, the results obtained from the simulations are presented and interpreted, allowing for a comprehensive understanding of the impact of different pValues and r values on the simulation's outcomes.

IV. RESULTS AND INTERPRETATION

The simulation was carried out 16 times, each iteration featuring distinct p values and r values. The research findings and their corresponding interpretations encompassed a comprehensive analysis of key metrics, namely Mean daily working time, Mean daily tour length, Mean number of left-over parcels, and delivery delays classified into categories of None, 1 day, 2 days, 3 days, 4 days, and ≥ 5 days.

It is essential to acknowledge that the deliveries were scheduled between 18:00 and 21:00, a timeframe prone to cancellations, customer non-responsiveness, and other real-life complexities inherent to late-night delivery services. Nonetheless, for the purposes of this study, these real-life complications were not factored into the analysis. Additionally, in the simulation's implementation, any delays extending beyond 12:00 AM were incorporated into the driver's working hours.

Moreover, the geographical locations of all customers were visually represented on a map, using numerical identifiers (1, 2, 3, etc.) to facilitate a more intuitive and lucid understanding of the delivery routes.

Through this meticulous examination, the research aimed to determine the most optimal bike and refine the delivery solution, taking into account the insights derived from the diverse combination of p and r values.

Note : Due to page limit constraints all figures are not included. But it can be accessed on jupyter file.

A. Pvalue = 0.22

Based on the simulation outcomes with a P-value of 0.22, optimal delivery efficiency is achieved with a 40 km range bike. The mean daily working time is calculated at 225.36, representing the average daily delivery task duration. The mean daily tour length covers approximately 39,730, signifying the average distance traveled during daily tours. On

average, 44.52 parcels remain undelivered daily. The number of leftover parcels (refer to Figure 5) exhibits minor spikes but remains generally acceptable.

Figure 4 provides cumulative data across all bike ranges for a comprehensive overview of the simulation results.

q value	0.2			
P value	0.22			
r Value(limit)	30000	35000	40000	45000
Mean daily working time	190.56	207.52	225.36	247.63
Mean daily tour length	29795	34781	39730	44704
Mean number of left over parcels	104.44	63.38	44.52	39.7
Delivery Delay				
None	12.37	15.05	20.81	22.67
1 day	11.29	16.13	25.35	28.14
2 days	11.29	14.7	27.94	28.49
3 days	10.57	16.04	9.53	9.86
4 days	12.46	14.07	5.64	4.66
>= 5 days	42.03	24	10.73	6.18

Fig. 4. Simulation summary for Pvalue as 0.22



Fig. 5. Daily leftover parcels for 40 KM bike range

B. Pvalue = 0.265

The data presented in Figure 6 indicates that a bike range of 40 km is the most suitable choice for our delivery needs. This bike range allows for a mean daily working time of 236.28 minutes, with a tour length of 39,797 and a mean of 64.14 leftover parcels. Furthermore, the delivery delay table shown in Figure 7 highlights that the daily leftover parcels are constant and don't have much spike.

Overall, the 40 km bike range offers a well-balanced solution, providing ample daily working hours and tour distance while maintaining a relatively low rate of delayed deliveries, making it the optimal choice for efficient and effective parcel distribution.

C. Pvalue = 0.285

According to the data depicted in Figure 8, a bike range of 45 km appears to be the most appropriate option for our delivery requirements. This particular bike range allows for an average daily working time of 268.94 minutes, covering a tour length of 44,783 and resulting in an average of 59.64 leftover parcels. Additionally, the delivery delay table presented in Figure 9 reveals that the leftover parcels are decreasing and no spike is seen.

q value	0.2			
P value	0.265			
r Value(limit)	30000	35000	40000	45000
Mean daily working time	197.02	217.12	236.28	264.42
Mean daily tour length	29803	34796.52	39797	44746
Mean number of left over parcels	130.8	94.3	64.14	61.28
Delivery Delay				
None	10.78	14.14	18.12	19.8
1 day	9.56	14.6	18.73	20.95
2 days	12	14.83	20.64	26.15
3 days	10.4	13.99	22.17	17.05
4 days	10.09	15.37	7.87	3.75
>= 5 days	47.17	27.06	12.46	12.31

Fig. 6. Simulation summary for Pvalue as 0.265

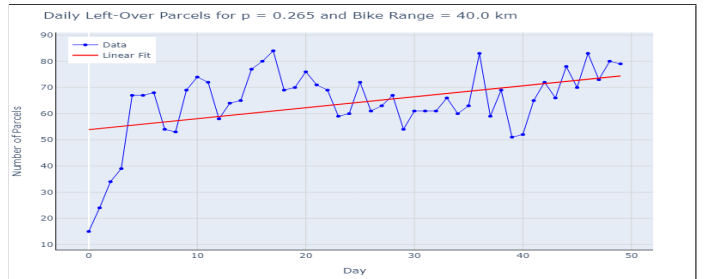


Fig. 7. Daily leftover parcels for 40 KM bike range

In summary, the 45 km bike range offers a well-balanced solution, providing ample daily working hours and tour distance, while simultaneously maintaining a relatively low rate of delayed deliveries. This makes it the most suitable and optimal choice for achieving efficient and effective parcel distribution.

q value	0.2			
P value	0.285			
r Value(limit)	30000	35000	40000	45000
Mean daily working time	209.74	233.4	246.6	268.94
Mean daily tour length	29721	34589.04	39817	44783
Mean number of left over parcels	151.9	115.94	84.08	59.64
Delivery Delay				
None	9.94	13.35	18.57	21.44
1 day	12.03	12.45	15.58	24.2
2 days	11.61	14.19	18.45	27.12
3 days	9.46	13.49	18.15	13.98
4 days	9.94	15.23	11.15	3.89
>= 5 days	47.01	31.29	17.52	9.04

Fig. 8. Simulation summary for Pvalue as 0.285

D. Pvalue = 0.35

The most fitting option for meeting our delivery requirements, as per the data presented in Figure 10, seems to be a bike range of 45 km. This particular range offers an average daily working time of 284.72 minutes, which can cover a tour length of 44775.62, resulting in an average of 84 parcels left over. The delivery delay table, as shown in Figure 11, indicates

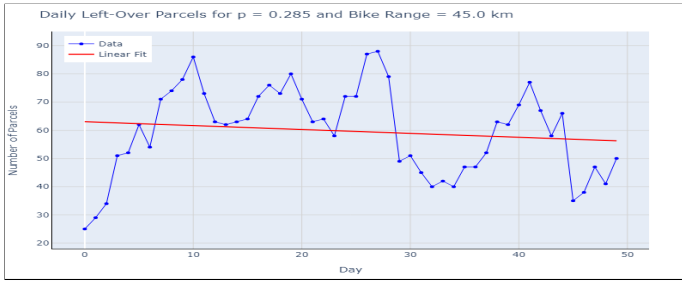


Fig. 9. Daily leftover parcels for 45 KM bike range

that only 12.85% of parcels are delayed beyond 5 days, a relatively low figure when compared to other bike ranges.

To sum up, the 45 km bike range is an ideal and well-balanced solution that provides sufficient daily working hours and tour distance, while also maintaining a relatively low rate of delayed deliveries. As such, it is the most appropriate and optimal choice for achieving efficient and effective parcel distribution.

q value	0.2			
P value	0.35			
r Value(limit)	30000	35000	40000	45000
Mean daily working time	227.78	240.66	268.32	284.72
Mean daily tour length	29748	34801.38	39799	44775.62
Mean number of left over parcels	193.48	146.2	122.2	84.28
Delivery Delay				
None	9.72	13.53	16.09	20.01
1 day	11.6	13.53	16.43	21.26
2 days	11.43	11.77	16.03	21.66
3 days	9.66	12.05	12.68	17.17
4 days	10.23	12.39	14.27	7.05
>= 5 days	47.36	36.73	24.5	12.85

Fig. 10. Simulation summary for Pvalue as 0.35

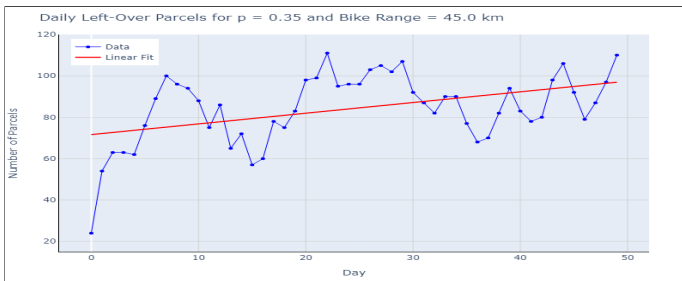


Fig. 11. Daily leftover parcels for 45 KM bike range

V. CONCLUSION

Based on 16 simulations with p-values ranging from 0.22 to 0.35, this study determined that the optimal bike range depends on the p-value. For p-values between 0.2 and 0.265, a 40 km bike range is deemed optimal. However, if the p-value exceeds 0.265, it is recommended to deploy a 45 km range bike to minimize leftover parcels and delivery delays. The study considered daily working time as a soft constraint,

as both bike ranges exceeded the given 3-hour time period on average.

To improve efficiency, the research suggests increasing the working time of the driver and selecting the appropriate bike range based on the specific p-value. Additionally, starting deliveries early may help reduce cancellations or non-responsiveness from customers. These insights could be valuable for We-Doo management to enhance their parcel distribution operations.

VI. REFLECTIONS AND FUTURE WORK

In future research, several avenues can be explored to further enhance the findings of this study. Firstly, conducting additional simulations with a broader range of p-values could provide a more comprehensive understanding of how different values impact bike range optimization and delivery performance. Moreover, considering variations in customer demand patterns and geographical factors could offer insights into tailoring the bike range selection to specific regions.

Additionally, incorporating real-world data and conducting field experiments would validate the simulation results and provide practical implications for implementing the suggested recommendations. Furthermore, exploring the integration of advanced technologies, such as route optimization algorithms and real-time tracking systems, could streamline the delivery process and reduce delivery delays further.

Furthermore, studying the impact of environmental factors, such as weather conditions and traffic patterns, on bike range selection and delivery efficiency could lead to more sustainable and environmentally friendly delivery strategies. Overall, these future research directions would contribute to the development of more robust and effective parcel distribution systems, benefiting both businesses and customers alike.

REFERENCES

- [1] Ravindra K. Ahuja, Kurt Mehlhorn, James Orlin, and Robert E. Tarjan. Faster algorithms for the shortest path problem. *J. ACM*, 37(2):213–223, apr 1990.
- [2] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968.
- [3] Zhibo He, Chenguang Liu, Xiumin Chu, Rudy R. Negenborn, and Qing Wu. Dynamic anti-collision a-star algorithm for multi-ship encounter situations. *Applied Ocean Research*, 118:102995, 2022.
- [4] Kairanbay Magzhan and Hajar Mat Jani. A review and evaluations of shortest path algorithms. *Int. J. Sci. Technol. Res.*, 2(6):99–104, 2013.
- [5] Gerrit Polder, Hans Hovens, and A. J. Zweers. Measuring shoot length of submerged aquatic plants using graph analysis. 2010.
- [6] Sungchul Han and Sukchan Kang. Optimizing all-pairs shortest-path algorithm using vector instructions. 2006.
- [7] Ismail H. Toroslu. Improving the floyd-warshall all pairs shortest paths algorithm. *CoRR*, abs/2109.01872, 2021.
- [8] Gözde Kizilates and Fidan Nuriyeva. On the nearest neighbor algorithms for the traveling salesman problem. In Dhinaharan Nagamalai, Ashok Kumar, and Annamalai Annamalai, editors, *Advances in Computational Science, Engineering and Information Technology*, pages 111–118, Heidelberg, 2013. Springer International Publishing.
- [9] Rio Ferdiani Harahap and Sawaluddin. Study vehicle routing problem using nearest neighbor algorithm. *Journal of Physics: Conference Series*, 2421, 2023.